

# GBIF Phoridae Snapshot Import Runbook

*hyparquet prescan -> DuckDB extraction -> API -> MySQL cache*

**Purpose:** safely rebuild or refresh the phorid.net GBIF Phoridae occurrence cache from a GBIF AWS Parquet snapshot, with a resumable prescan manifest and a filtered DuckDB import queue.

## One-page operating summary

- Use hyparquet first to prescan the GBIF Parquet snapshot and create a CSV audit log of which part files contain Phoridae.
- Deduplicate the append-only prescan log into a clean one-row-per-file CSV.
- Register only successful, Phoridae-positive Parquet files into gbif\_import\_file.
- Run the DuckDB importer with --skip-register so it processes only the filtered queue.
- Use --max-attempts 1 for the main pass so failures are skipped and collected for a later retry pass.
- Use longer DuckDB HTTP timeout settings, currently manually set in the script, because larger Parquet parts can time out at the default.
- For future snapshots, you can upsert into the existing occurrence table and delete old snapshot rows only after the new import completes.

## System architecture

```
GBIF AWS snapshot
s3://gbif-open-data-us-east-1/occurrence/YYYY-MM-DD/occurrence.parquet/*
    |
    | 1. hyparquet prescan, Node only
    v
Prescan CSV audit log + clean manifest
    |
    | 2. register matching files only
    v
phorid.net API: /api/gbif-import/files/register
    |
    v
MySQL: gbif_import_file filtered extraction queue
    |
    | 3. DuckDB importer with --skip-register
    v
DuckDB reads selected S3 Parquet files and writes temporary CSV rows
    |
    | 4. Node posts occurrence batches
    v
phorid.net API: /api/gbif-import/occurrences/batch
    |
    v
MySQL: gbif_occurrence_phoridae
```

## Important paths and knobs

| Item                  | Value / meaning                                             |
|-----------------------|-------------------------------------------------------------|
| Local tooling folder  | /Users/Deinonychus/Documents/phorid-net-3/gbif_sync_tooling |
| Backend folder        | /Users/Deinonychus/Documents/phorid-net-3/backend           |
| Snapshot date example | 2026-06-01                                                  |
| GBIF family           | Phoridae                                                    |
| Hosted API base       | https://api.phorid.net/api/gbif-import                      |
| Local API base        | http://localhost:8080/api/gbif-import                       |
| MySQL database        | johnhash_phorid_research                                    |

| Item                   | Value / meaning                  |
|------------------------|----------------------------------|
| Import queue table     | gbif_import_file                 |
| Occurrence cache table | gbif_occurrence_phoridae         |
| Prescan script         | scripts/prescan-hyparquet.js     |
| Register script        | scripts/register-from-prescan.js |
| DuckDB sync script     | scripts/sync-gbif-to-api.js      |

**Security note: never paste your GBIF\_IMPORT\_TOKEN or database password into this runbook. Keep them in .env / cPanel environment variables and out of Git.**

## Before running: environment and health checks

### Check the Mac tooling folder

```
cd /Users/Deinonychus/Documents/phorid-net-3/gbif_sync_tooling
pwd
```

### Check the .env target

```
# Local backend target, useful for testing:
GBIF_IMPORT_API_BASE=http://localhost:8080/api/gbif-import

# Hosted backend target, useful for real import:
GBIF_IMPORT_API_BASE=https://api.phorid.net/api/gbif-import

GBIF_IMPORT_TOKEN=YOUR_LONG_SECRET_TOKEN
GBIF_AWS_REGION=us-east-1
GBIF_SNAPSHOT_DATE=2026-06-01
GBIF_FAMILY=Phoridae
DUCKDB_BIN=./bin/duckdb-docker
```

### Check Docker and DuckDB wrapper

```
open -a Docker

docker info

./bin/duckdb-docker -c "SELECT 1 AS duckdb_ok;"
```

### Check API health

```
node scripts/sync-gbif-to-api.js \
  --skip-register \
  --max-files 1 \
  --dry-run
```

## Stage 1 - Prescan the GBIF snapshot with hyparquet

- This is Node-only and does not use Docker, DuckDB, the API, or MySQL.
- It reads only the family column from each Parquet part file.
- It appends one result row per attempt to the full CSV audit log.
- On resume, ok=1 rows are skipped, failed-only rows are retried, and missing rows are scanned.

### Full prescan command

```
cd /Users/Deinonychus/Documents/phorid-net-3/gbif_sync_tooling
```

```
node scripts/prescan-hyparquet.js \
--concurrency 4 \
--resume \
--max-consecutive-errors 50 \
--output tmp/hyparquet_prescan_2026-06-01_Phoridae_full.csv
```

## If internet drops or the process stops

Run the exact same prescan command again. The script will skip successful ok=1 files and retry files that only have ok=0 rows.

## Useful prescan checks

```
# Unique successful files
awk -F',' 'NR>1 && $7=="1" { seen[$1]=1 } END { print length(seen) }' \
tmp/hyparquet_prescan_2026-06-01_Phoridae_full.csv

# Failed-only files still needing retry
awk -F',' '
NR>1 {
  if ($7=="1") ok[$1]=1;
  else fail[$1]=1;
}
END {
  count=0;
  for (k in fail) if (!(k in ok)) count++;
  print count;
}' tmp/hyparquet_prescan_2026-06-01_Phoridae_full.csv

# Unique files with Phoridae
awk -F',' 'NR>1 && $7=="1" && $2=="1" { seen[$1]=1 } END { print length(seen) }' \
tmp/hyparquet_prescan_2026-06-01_Phoridae_full.csv
```

## Stage 2 - Create the clean prescan CSV

The full CSV is an append-only audit log, so the same file key can appear once as failed and later as successful. The clean CSV is a deduped one-row-per-file manifest used for registration.

```
python3 - <<'PY'
import csv

input_path = "tmp/hyparquet_prescan_2026-06-01_Phoridae_full.csv"
output_path = "tmp/hyparquet_prescan_2026-06-01_Phoridae_clean.csv"

best = {}
with open(input_path, newline="") as f:
    reader = csv.DictReader(f)
    fieldnames = reader.fieldnames
    for row in reader:
        key = row["file_key"]
        if row["ok"] == "1":
            best[key] = row
        elif key not in best:
            best[key] = row
        elif best[key].get("ok") != "1":
            best[key] = row

with open(output_path, "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    for key in sorted(best):
        writer.writerow(best[key])

matched = [r for r in best.values() if r["ok"] == "1" and r["has_match"] == "1"]
failed = [r for r in best.values() if r["ok"] != "1"]
print(f"Wrote {len(best)} deduped rows to {output_path}")
print(f"Files with Phoridae: {len(matched)}")
print(f"Failed-only files: {len(failed)}")
print(f"Predicted Phoridae rows: {sum(int(r['match_count'] or 0) for r in matched)}")
PY
```

## Stage 3 - Decide whether to clear database state

### First full rebuild or when you want a clean cache

For the initial rebuild, clearing both the occurrence table and import queue is simplest. This removes partial earlier imports and old unfiltered queue entries.

```
SET FOREIGN_KEY_CHECKS=0;

DELETE FROM gbif_occurrence_phoridae;
DELETE FROM gbif_import_file;
DELETE FROM gbif_import_run;

ALTER TABLE gbif_occurrence_phoridae AUTO_INCREMENT = 1;
ALTER TABLE gbif_import_file AUTO_INCREMENT = 1;
ALTER TABLE gbif_import_run AUTO_INCREMENT = 1;

SET FOREIGN_KEY_CHECKS=1;
```

### Future snapshot refresh without wiping occurrence rows first

For a future snapshot, do not clear `gbif_occurrence_phoridae` at the beginning if you want to keep the current cache working during the new import. Clear only the import queue for the new snapshot, then upsert the new data. After the new snapshot is fully imported and checked, delete old snapshot rows.

```
-- Example for a future 2026-12-01 snapshot.
DELETE FROM gbif_import_file
WHERE snapshot_date = '2026-12-01';

DELETE FROM gbif_import_run
WHERE snapshot_date = '2026-12-01';
```

**Why not TRUNCATE `gbif_import_run`? MySQL can block TRUNCATE on a table referenced by a foreign key. DELETE is safer here.**

## Stage 4 - Register only Phoridae-positive files

This fills `gbif_import_file` with only files from the clean prescan CSV where `ok=1`, `has_match=1`, and `match_count > 0`.

```
node scripts/register-from-prescan.js \
  --input tmp/hyparquet_prescan_2026-06-01_Phoridae_clean.csv \
  --snapshot 2026-06-01 \
  --batch-size 500
```

Then check status. The import queue should show pending files, and `occurrence_count` should be 0 for a clean rebuild or the previous snapshot count if doing a non-destructive refresh.

```
# In a browser or curl, with your token:
https://api.phorid.net/api/gbif-import/status?token=YOUR_TOKEN&snapshot_date=2026-06-01
```

**Important: from this point on, the DuckDB import command must use `--skip-register`. Do not let the DuckDB script register all S3 files again.**

## Stage 5 - Run DuckDB extraction to API/MySQL

The DuckDB sync script asks the API for pending files, reads each selected Parquet part, extracts Phoridae rows to a temporary CSV, parses those rows in Node, and posts batches to the API. The API upserts rows into MySQL by `gbifid`.

### Stable main-pass command

```
node scripts/sync-gbif-to-api.js \
```

```
--skip-register \
--max-files 250 \
--max-attempts 1 \
--batch-size 250 \
--max-consecutive-failures 5
```

## Longer daytime run once stable

```
node scripts/sync-gbif-to-api.js \
--skip-register \
--max-files 500 \
--max-attempts 1 \
--batch-size 250 \
--max-consecutive-failures 5
```

- --skip-register means use the already-filtered gbif\_import\_file queue.
- --max-files controls how many queued files to process in this run. It does not increase API batch size.
- --max-attempts 1 is best for the main pass: process fresh pending files and leave failures for a later retry pass.
- --batch-size 250 controls how many occurrence rows are posted per API request.
- --max-consecutive-failures 5 stops the run if S3/Docker/network failures start happening repeatedly.

## DuckDB timeout setting

You manually set the DuckDB HTTP timeout in the script. Keep that change documented. The important generated SQL settings are:

```
SET http_timeout = 900;
SET http_retries = 10;
```

The 900-second timeout is important because larger Parquet part files, often over 50 MB, can fail with DuckDB httpfs default timeouts.

## Stage 6 - Monitor progress

### API status endpoint

```
https://api.phorid.net/api/gbif-import/status?token=YOUR_TOKEN&snapshot_date=2026-06-01
```

### Read the status fields

| Item             | Value / meaning                                                                                                                       |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| complete         | Files fully processed by DuckDB and marked complete.                                                                                  |
| pending          | Registered files not yet attempted or reset to retry.                                                                                 |
| running          | Files currently marked as in progress. If the script is not running, these are stale.                                                 |
| failed           | Files that errored. Leave them alone during the main pass, then retry later with higher max-attempts.                                 |
| extracted_count  | Sum of rows reported complete for completed files.                                                                                    |
| occurrence_count | Rows currently in gbif_occurrence_phoridae. This can be higher than extracted_count if some rows inserted before a file later failed. |
| top_genera       | Quick sanity check. Null genus can be legitimate for BOLD/barcoding or family-level records.                                          |

## Stage 7 - What to do when something fails

### If you see many consecutive S3 timeouts

- Stop the run with Ctrl+C if the safety stop has not already stopped it.
- Confirm Docker Desktop is running and your internet is stable.
- Keep the 900-second DuckDB HTTP timeout.
- Resume the main pass later with --max-attempts 1. Already failed files will be skipped until the retry pass.

### If files are stuck as running after an interrupted process

If the script is definitely not running, rows with status = running are stale. You have two safe reset options.

Option A - Document the interruption and retry later as failures. This preserves attempt\_count history:

```
UPDATE gbif_import_file
SET
  status = 'failed',
  finished_at = NOW(),
  error_message = 'Manually reset from running after interrupted process.'
WHERE snapshot_date = '2026-06-01'
  AND status = 'running';
```

Option B - Put them back into the main pending queue as fresh attempts. Use this when you want them picked up by --max-attempts 1:

```
UPDATE gbif_import_file
SET
  status = 'pending',
  attempt_count = 0,
  started_at = NULL,
  finished_at = NULL,
  error_message = NULL
WHERE snapshot_date = '2026-06-01'
  AND status = 'running';
```

### Retry failed files after the main pass

After pending reaches zero, retry failed files with higher max-attempts and smaller batches.

```
node scripts/sync-gbif-to-api.js \
  --skip-register \
  --max-files 25 \
  --max-attempts 3 \
  --batch-size 100 \
  --max-consecutive-failures 5
```

If some files still fail after multiple attempts, list them:

```
SELECT
  file_key,
  attempt_count,
  error_message
FROM gbif_import_file
WHERE snapshot_date = '2026-06-01'
  AND status = 'failed'
ORDER BY file_key;
```

### Reset all failed files for a fresh retry

```
UPDATE gbif_import_file
SET
  status = 'pending',
  attempt_count = 0,
  started_at = NULL,
  finished_at = NULL,
  error_message = NULL
WHERE snapshot_date = '2026-06-01'
  AND status = 'failed';
```

## Stage 8 - Final checks after import completes

Run these checks when pending = 0 and failed files have either completed or been deliberately accepted as unresolved.

### File status summary

```
SELECT
  status,
  COUNT(*) AS file_count,
  COALESCE(SUM(extracted_count), 0) AS extracted_count
FROM gbif_import_file
WHERE snapshot_date = '2026-06-01'
GROUP BY status
ORDER BY status;
```

### Occurrence count

```
SELECT COUNT(*) AS occurrence_count
FROM gbif_occurrence_phoridae
WHERE snapshot_date = '2026-06-01';
```

### Taxon rank sanity check

```
SELECT
  taxon_rank,
  COUNT(*) AS row_count
FROM gbif_occurrence_phoridae
WHERE snapshot_date = '2026-06-01'
GROUP BY taxon_rank
ORDER BY row_count DESC;
```

### Top genera sanity check

```
SELECT
  genus,
  COUNT(*) AS row_count
FROM gbif_occurrence_phoridae
WHERE snapshot_date = '2026-06-01'
GROUP BY genus
ORDER BY row_count DESC
LIMIT 50;
```

## Six-month refresh workflow for a new snapshot

Use this workflow when a future GBIF snapshot such as 2026-12-01 is available and you want to refresh the cache without wiping the current occurrence table first.

1. Change GBIF\_SNAPSHOT\_DATE in gbif\_sync\_tooling/.env to the new snapshot date, for example 2026-12-01.
2. Run the hyparquet prescan for the new snapshot into a new full CSV file.
3. Create the clean deduped CSV for the new snapshot.
4. Clear only gbif\_import\_file / gbif\_import\_run rows for the new snapshot, not the occurrence table.
5. Register only Phoridae-positive files from the new clean CSV.
6. Run DuckDB import with --skip-register. Existing gbifid rows will be updated; new gbifid rows will be inserted.
7. After the new snapshot is complete and checked, delete rows whose snapshot\_date is not the new snapshot if you want a current-only cache.

### Future snapshot commands template

```
# 1. Prescan new snapshot
node scripts/prescan-hyparquet.js \
  --snapshot 2026-12-01 \
  --concurrency 4 \
  --resume \
  --max-consecutive-errors 50 \
  --output tmp/hyparquet_prescan_2026-12-01_Phoridae_full.csv

# 2. Create clean CSV by adapting the dedupe script paths:
# input = tmp/hyparquet_prescan_2026-12-01_Phoridae_full.csv
```

```
# output = tmp/hyparquet_prescan_2026-12-01_Phoridae_clean.csv

# 3. Clear only the new snapshot import queue in MySQL
DELETE FROM gbif_import_file WHERE snapshot_date = '2026-12-01';
DELETE FROM gbif_import_run WHERE snapshot_date = '2026-12-01';

# 4. Register the filtered queue
node scripts/register-from-prescan.js \
  --input tmp/hyparquet_prescan_2026-12-01_Phoridae_clean.csv \
  --snapshot 2026-12-01 \
  --batch-size 500

# 5. Main DuckDB import pass
node scripts/sync-gbif-to-api.js \
  --skip-register \
  --snapshot 2026-12-01 \
  --max-files 250 \
  --max-attempts 1 \
  --batch-size 250 \
  --max-consecutive-failures 5
```

## After successful future import

Only after you are comfortable that the new snapshot imported correctly, remove rows not touched by the new snapshot:

```
DELETE FROM gbif_occurrence_phoridae
WHERE snapshot_date <> '2026-12-01';
```

**This cleanup works only if the upsert updates snapshot\_date on duplicate gbifid. Confirm the API upsert includes snapshot\_date = VALUES(snapshot\_date).**

## Operational notes and cautions

- The GBIF cache is rebuildable. Your original research/specimen/species-description data should remain separate and backed up.
- During a future non-destructive refresh, the occurrence table can temporarily contain a mix of old and new snapshot rows until cleanup.
- If public API endpoints read the table during refresh, consider filtering by a known current snapshot date or adding a current snapshot setting later.
- The temporary DuckDB CSV bridge is acceptable for this proven workflow. A future hyparquet-native importer could skip DuckDB/Docker and stream directly to the API.
- Null genus is expected for many barcoding or higher-rank GBIF records and is not automatically a problem.
- Keep the prescan full CSV as the audit log. Keep the clean CSV as the manifest used to register the extraction queue.

## Command quick reference

```
# Prescan, resumable
node scripts/prescan-hyparquet.js \
  --concurrency 4 \
  --resume \
  --max-consecutive-errors 50 \
  --output tmp/hyparquet_prescan_2026-06-01_Phoridae_full.csv

# Register filtered queue
node scripts/register-from-prescan.js \
  --input tmp/hyparquet_prescan_2026-06-01_Phoridae_clean.csv \
  --snapshot 2026-06-01 \
  --batch-size 500

# Main import pass
node scripts/sync-gbif-to-api.js \
  --skip-register \
  --max-files 250 \
  --max-attempts 1 \
  --batch-size 250 \
  --max-consecutive-failures 5
```



```
# Retry failed files
node scripts/sync-gbif-to-api.js \
  --skip-register \
  --max-files 25 \
  --max-attempts 3 \
  --batch-size 100 \
  --max-consecutive-failures 5
```